

SocialHub — Production Deployment Guide (AWS EC2)

A complete, self-contained runbook. Follow top to bottom to deploy the app on a single Ubuntu EC2 instance with Nginx + PM2 + Docker Postgres + HTTPS. Replace <PLACEHOLDERS> with your values.

1. Architecture

A monorepo with two apps: **client/** (React + TypeScript + Vite) and **server/** (Node + Express + TypeScript, PostgreSQL via Drizzle ORM). Everything runs on one EC2 box under a single origin:

- **Nginx** — serves the built client (static files) from `/var/www/socialhub` and reverse-proxies `/api` → Node on `127.0.0.1:4000`; terminates TLS.
- **Node API** — runs under **PM2** (auto-restart + boot persistence). An in-process scheduler auto-publishes due posts.
- **PostgreSQL** — runs via the repo's `docker-compose.yml` on the same box (port 5432 NOT exposed publicly).
- **Certbot** — issues/auto-renews the Let's Encrypt certificate.

Single origin means the client's relative `/api` calls and auth cookies work with no CORS setup.

2. Prerequisites

- Code pushed to a **GitHub repo** (confirm `server/.env` and `*.pem` are gitignored — never commit secrets).
- A **domain** you control (example uses subdomain `app.example.com`).
- OAuth app credentials (YouTube/Google, LinkedIn, X), a **Cloudinary** account, a **Razorpay** key pair, and an SMTP account (e.g. a Gmail **App Password**) — all optional; features stay disabled until their env vars are set.

3. Launch the EC2 instance

- **AMI:** Ubuntu Server 24.04 LTS · **Type:** t3.small (t2.micro works but add swap) · **Key pair:** download the `.pem`.
- **Elastic IP:** allocate + associate so the public IP is stable for DNS.
- **Security Group — inbound:**

Port	Source	Purpose
22	Your IP	SSH
80	0.0.0.0/0	HTTP (Certbot + redirect)
443	0.0.0.0/0	HTTPS

Do NOT open 5432 — Postgres must stay private to the instance.

4. DNS

At your DNS provider add an **A record:** name `app` (the subdomain) → your **Elastic IP**. Verify: `dig +short app.example.com` returns the IP.

5. SSH in & install dependencies

```
ssh -i your-key.pem ubuntu@<ELASTIC_IP>

sudo apt-get update && sudo apt-get -y upgrade
# Node 20 LTS + git + nginx
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs git nginx
# Docker + compose plugin
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER # log out/in so docker works without sudo
# Certbot + PM2
sudo apt-get install -y certbot python3-certbot-nginx
sudo npm install -g pm2
```

t2.micro only — add 2 GB swap so builds don't OOM:

```
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

6. Clone the repo

```
cd ~
git clone https://github.com/<you>/<repo>.git social-hub
cd social-hub # path used throughout: /home/ubuntu/social-hub
```

7. Start PostgreSQL (Docker)

Compose reads `POSTGRES_PASSWORD` from a **root** `.env` (gitignored):

```
echo "POSTGRES_PASSWORD=$(openssl rand -hex 16)" > /home/ubuntu/social-hub/.env
cat .env # copy this password — reused in DATABASE_URL below
docker compose up -d
docker compose ps # postgres should be "healthy"
```

8. Create `server/.env` (production)

Generate fresh secrets with `openssl rand -hex 32`. Full reference:

```
NODE_ENV=production
PORT=4000
CLIENT_ORIGIN=https://app.example.com
API_BASE_URL=https://app.example.com

# Same password as step 7
DATABASE_URL=postgres://app:<POSTGRES_PASSWORD>@localhost:5432/socialhub

JWT_ACCESS_SECRET=<openssl rand -hex 32>
JWT_REFRESH_SECRET=<openssl rand -hex 32>
ACCESS_TOKEN_TTL_SECONDS=900
REFRESH_TOKEN_TTL_DAYS=30

COOKIE_SECURE=true
COOKIE_SAMESITE=lax

ENCRYPTION_KEY=<openssl rand -hex 32> # 64 hex chars; encrypts stored OAuth tokens
OAUTH_STATE_SECRET=<openssl rand -hex 32>

# OAuth apps (optional)
YOUTUBE_CLIENT_ID=... YOUTUBE_CLIENT_SECRET=...
LINKEDIN_CLIENT_ID=... LINKEDIN_CLIENT_SECRET=...
LINKEDIN_API_VERSION=202506
X_CLIENT_ID=... X_CLIENT_SECRET=...

# Cloudinary — media + avatar uploads (optional)
CLOUDINARY_CLOUD_NAME=... CLOUDINARY_API_KEY=... CLOUDINARY_API_SECRET=...

# Razorpay — Pro subscriptions (optional). Key id is public; secret stays server-side.
RAZORPAY_KEY_ID=... RAZORPAY_KEY_SECRET=...

# Email/SMTP — password-reset mail (optional). Gmail: host smtp.gmail.com, port 465, App Password.
EMAIL_SERVER_HOST=smtp.gmail.com
EMAIL_SERVER_PORT=465
EMAIL_SERVER_USER=you@gmail.com
EMAIL_SERVER_PASSWORD=<gmail app password>
EMAIL_FROM=SocialHub <you@gmail.com>
```

```
SCHEDULER_INTERVAL_SECONDS=30
```

Don't change `ENCRYPTION_KEY` after connections exist — stored OAuth tokens would fail to decrypt. The server exits on any invalid/missing required env var.

9. Build, migrate & run the API

```
cd /home/ubuntu/social-hub/server
npm install
npm run build          # tsc -> dist/
npm run db:migrate     # applies drizzle/*.sql (schema + all migrations)
pm2 start ecosystem.config.cjs
pm2 save && pm2 startup  # run the command pm2 startup prints (boot persistence)
curl http://localhost:4000/api/health # {"status":"ok"}
```

10. Build the client & publish

```
cd /home/ubuntu/social-hub/client
npm install
npm run build          # -> client/dist
sudo mkdir -p /var/www/socialhub
sudo cp -r dist/. /var/www/socialhub/
```

11. Configure Nginx

The repo ships a config at `deploy/nginx/socialhub.conf` (SPA fallback + `/api` proxy + web root). Update its `server_name` to your domain, then:

```
sudo cp /home/ubuntu/social-hub/deploy/nginx/socialhub.conf /etc/nginx/sites-available/socialhub
sudo ln -s /etc/nginx/sites-available/socialhub /etc/nginx/sites-enabled/socialhub
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
```

If writing the config by hand, it must: (a) set `root /var/www/socialhub;`, (b) `location / { try_files $uri /index.html; }` for SPA routes, (c) `location /api/ { proxy_pass http://127.0.0.1:4000; }` with proxy headers.

12. Enable HTTPS

```
sudo certbot --nginx -d app.example.com # choose the HTTP->HTTPS redirect option
```

Certbot rewrites the Nginx config for TLS and auto-renews via a systemd timer.

13. Register production OAuth redirect URIs (critical for social connect)

In each provider console add (in addition to any localhost URIs):

- **Google/YouTube:** `https://app.example.com/api/connections/oauth/youtube/callback` (also add the domain to the OAuth consent screen; publish it to Production to avoid the "unverified app" warning).
- **LinkedIn:** `https://app.example.com/api/connections/oauth/linkedin/callback`
- **X:** `https://app.example.com/api/connections/oauth/x/callback`

14. Verify

Open `https://app.example.com` → register → create a workspace → Connections → connect a platform → compose & publish. Test Billing with a Razorpay test card, and Forgot Password to confirm SMTP.

15. Redeploy after a code change

```

cd /home/ubuntu/social-hub
git pull
cd server && npm install && npm run build && npm run db:migrate && pm2 restart socialhub-api
cd ../client && npm install && npm run build && sudo cp -r dist/. /var/www/socialhub/
# reload nginx only if deploy/nginx/socialhub.conf changed

```

16. Troubleshooting

Symptom	Cause / fix
Browser timeout	Security Group missing inbound 80/443 (timeout = silently dropped; "refused" = nothing listening).
502 Bad Gateway	API down: <code>pm2 logs socialhub-api</code> . Usually a bad <code>DATABASE_URL</code> or missing required env (app exits on invalid env).
DB connection refused	<code>docker compose ps</code> ; ensure <code>DATABASE_URL</code> password matches the root <code>.env</code> .
OAuth redirect_uri mismatch	Provider's registered URI must match the callback in step 13 exactly.
Disk full	<code>sudo apt-get clean; sudo docker system prune -f; npm cache clean --force</code> . Long-term: grow the EBS volume + <code>growpart</code> / <code>resize2fs</code> .
Backup DB	<code>docker exec socialhub-postgres pg_dump -U app socialhub > backup.sql</code>